



CELEBAL
TECHNOLOGIES

**DELTA LAKE INCREMENTAL DATA PROCESSING
USING PYSPARK**

Celebal Summer Internship 2026

Week 7 Assignment

**Delta Lake Incremental Data Processing Using
MERGE Operation**

Dataset: Sample Superstore Dataset

Submission Date:

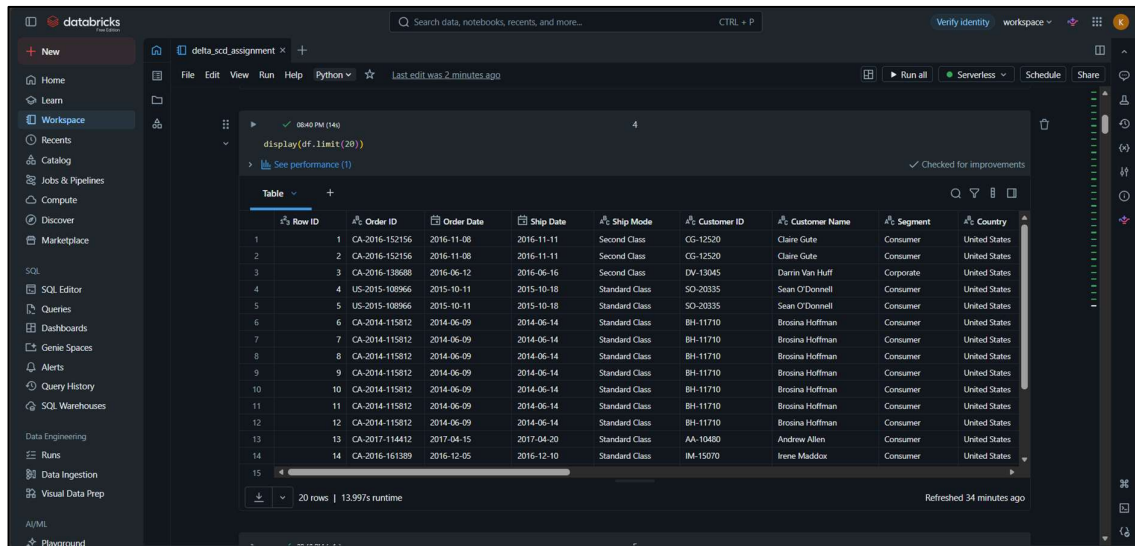
05-07-2026

Prepared By:

Keshav Kumar sharma

Loading and Displaying the Customer Dataset

The Sample Superstore dataset was successfully loaded into a PySpark DataFrame and the first 20 records were displayed to verify that the data was imported correctly.

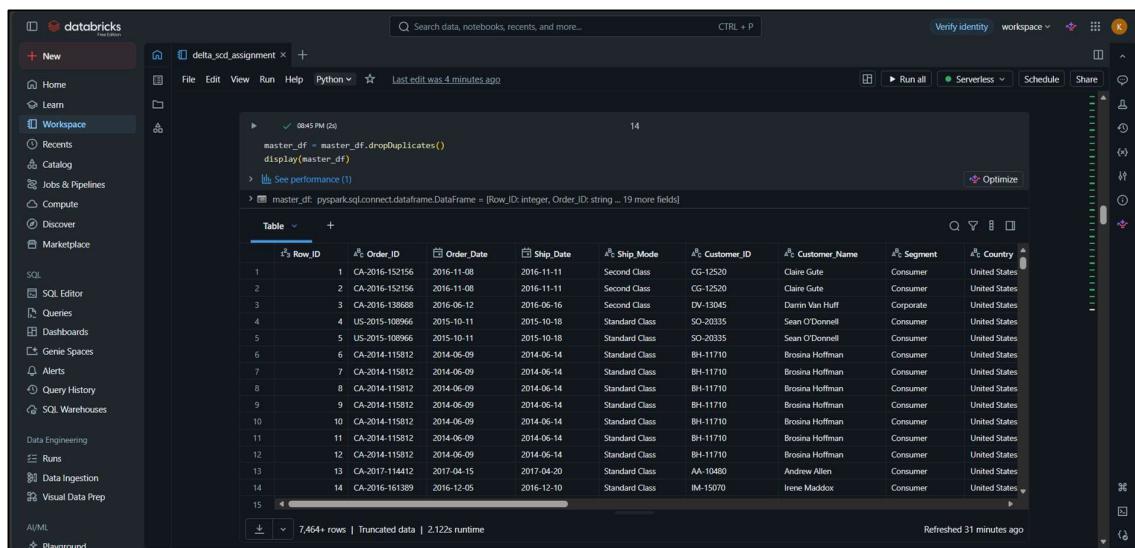


The screenshot shows the Databricks workspace interface. The notebook code cell contains the command `display(df.limit(20))`. Below the code, a table displays the first 20 rows of the dataset. The table has 9 columns: Row ID, Order ID, Order Date, Ship Date, Ship Mode, Customer ID, Customer Name, Segment, and Country. The data shows various orders from 2014 to 2017, with customers like Claire Gule, Darrin Van Huff, Sean O'Donnell, Brosina Hoffman, and Andrew Allen.

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country
1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gule	Consumer	United States
2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gule	Consumer	United States
3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate	United States
4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States
5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States
6	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States
7	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States
8	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States
9	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States
10	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States
11	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States
12	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States
13	CA-2017-114412	2017-04-15	2017-04-20	Standard Class	AA-10480	Andrew Allen	Consumer	United States
14	CA-2016-161389	2016-12-05	2016-12-10	Standard Class	IM-15070	Irene Maddox	Consumer	United States

Step 2: Data Cleaning and Removing Duplicate Records

Duplicate records were removed from the dataset using the `dropDuplicates()` function to ensure data consistency before storing the data in the Delta Table.



The screenshot shows the Databricks workspace interface. The notebook code cell contains the command `master_df = master_df.dropDuplicates()` followed by `display(master_df)`. Below the code, a table displays the first 14 rows of the dataset after removing duplicates. The table has 9 columns: Row ID, Order ID, Order Date, Ship Date, Ship Mode, Customer ID, Customer Name, Segment, and Country. The data shows various orders from 2014 to 2017, with customers like Claire Gule, Darrin Van Huff, Sean O'Donnell, Brosina Hoffman, and Andrew Allen.

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country
1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gule	Consumer	United States
2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gule	Consumer	United States
3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate	United States
4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States
5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States
6	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States
7	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States
8	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States
9	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States
10	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States
11	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States
12	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States
13	CA-2017-114412	2017-04-15	2017-04-20	Standard Class	AA-10480	Andrew Allen	Consumer	United States
14	CA-2016-161389	2016-12-05	2016-12-10	Standard Class	IM-15070	Irene Maddox	Consumer	United States

Step 3: Data Validation and Data Type Conversion

The cleaned dataset was validated by checking the total number of records after removing duplicates. The Sales column was converted to the Double data type to ensure compatibility with Delta Lake operations and accurate numerical processing.

The screenshot shows the Databricks workspace interface. The left sidebar contains navigation options like Home, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Spaces, Alerts, Query History, SQL Warehouses, Data Engineering, Runs, Data Ingestion, Visual Data Prep, AI/ML, and Playground. The main area displays a notebook with the following code and output:

```
master_df = master_df.withColumn("Sales", col("Sales").cast("double"))
display(master_df.limit(10))
```

The output shows a table with 10 rows and 10 columns: Row_ID, Order_ID, Order_Date, Ship_Date, Ship_Mode, Customer_ID, Customer_Name, Segment, and Country. The Sales column is now of type Double.

Row_ID	Order_ID	Order_Date	Ship_Date	Ship_Mode	Customer_ID	Customer_Name	Segment	Country
1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12620	Claire Gule	Consumer	United States
2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gule	Consumer	United States
3	CA-2016-130688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate	United States
4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States
5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States
6	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States
7	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States
8	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States
9	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States
10	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States

Step 4: Updating Existing Customer Records

Existing customer records were updated by modifying the Order_ID, Customer_Name, and Sales values for selected Row_IDs. The updated records were displayed to verify that the changes were applied successfully before creating the incremental dataset.

The screenshot shows the Databricks workspace interface. The left sidebar contains navigation options like Home, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Spaces, Alerts, Query History, SQL Warehouses, Data Engineering, Runs, Data Ingestion, Visual Data Prep, AI/ML, and Playground. The main area displays a notebook with the following code and output:

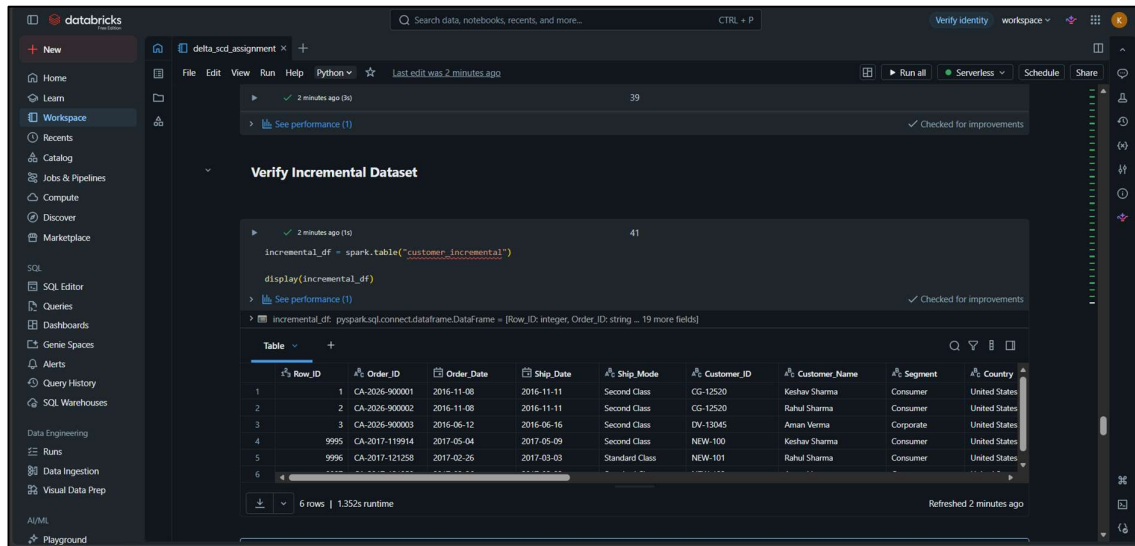
```
updated_df = pyspark.sql.connect.dataframe.DataFrame([Row_ID: integer, Order_ID: string ... 19 more fields])
display(updated_df.select("Row_ID", "Sales", "Customer_Name"))
```

The output shows a table with 3 rows and 3 columns: Row_ID, Sales, and Customer_Name. The Sales column is of type Double.

Row_ID	Sales	Customer_Name
1	361.96	Keshav Sharma
2	831.94	Rahul Sharma
3	114.62	Aman Verma

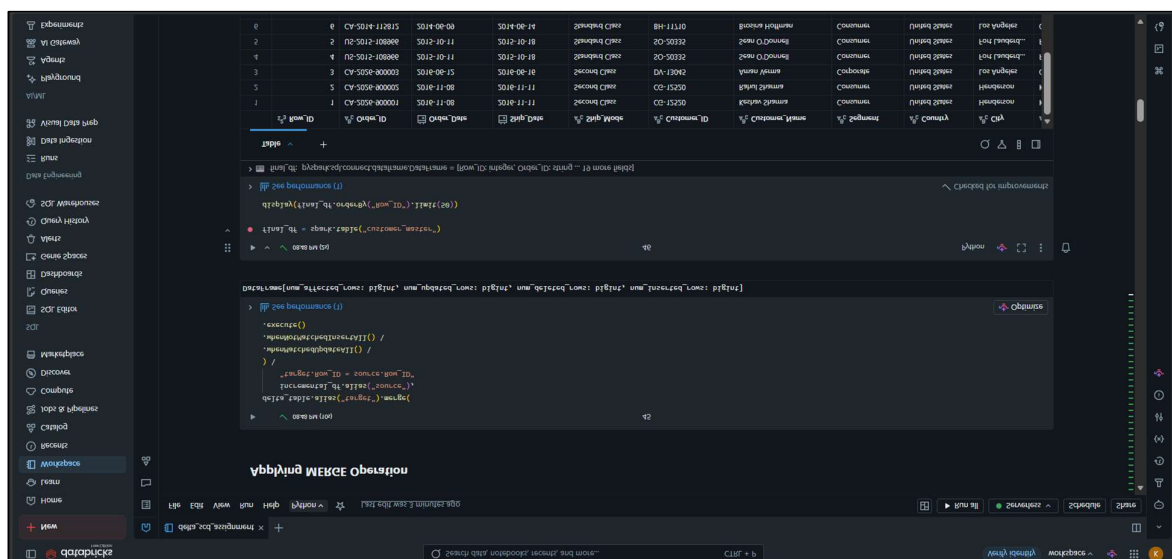
Step 5: Creating and Verifying the Incremental Dataset

An incremental dataset containing both updated records and new customer records was created and stored as a Delta Table. The dataset was then displayed to verify that all changes were correctly prepared before performing the MERGE operation.



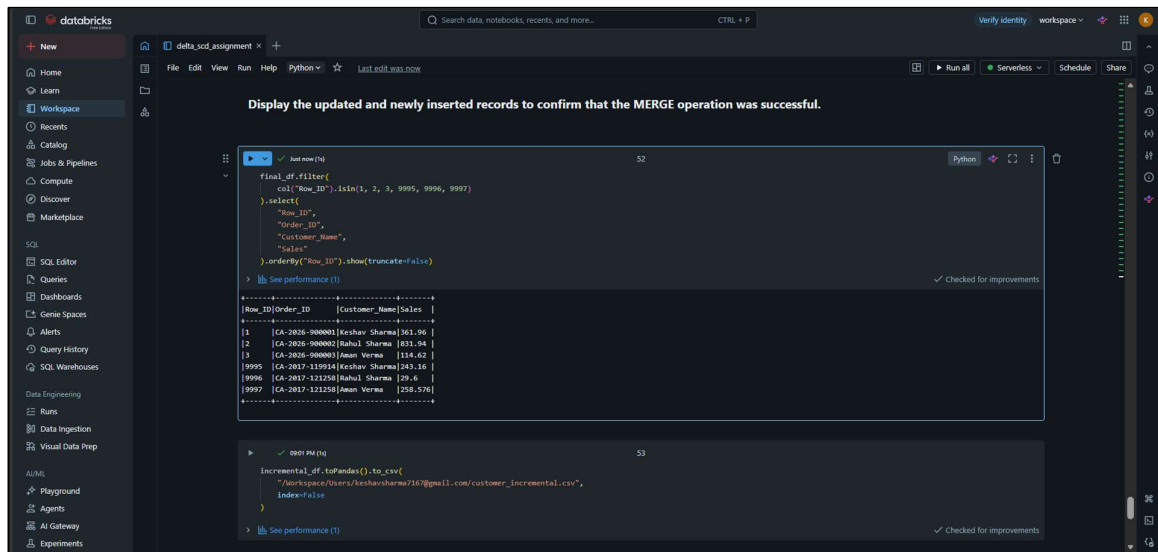
Step 6: Applying the Delta Lake MERGE Operation

The **MERGE** operation was performed on the Delta Table using the incremental dataset. Existing records were updated, new records were inserted, and the final Delta Table was displayed to verify that the merge was executed successfully.



Step 7: Validating the Final MERGE Results

The final Delta Table was validated by displaying both the updated records (Row_ID 1, 2, 3) and the newly inserted records (Row_ID 9995, 9996, 9997). The output confirms that the MERGE operation successfully updated existing records and inserted new records without creating duplicate Row_IDs.



The screenshot shows the Databricks workspace interface. The main panel displays a Python notebook with a code cell that filters the final Delta table for Row_IDs 1, 2, 3, 9995, 9996, and 9997. The output shows a table with columns Row_ID, Order_ID, Customer_Name, and Sales. The table contains 6 rows, confirming that the MERGE operation was successful and no duplicate Row_IDs were created.

```
final_df.filter(
    col("Row_ID").isin(1, 2, 3, 9995, 9996, 9997)
).select(
    "Row_ID",
    "Order_ID",
    "Customer_Name",
    "Sales"
).orderBy("Row_ID").show(truncate=False)
```

Row_ID	Order_ID	Customer_Name	Sales
1	CA-2026-980001	Keshav Sharma	361.96
2	CA-2026-980002	Rahul Sharma	831.94
3	CA-2026-980003	Anam Verma	114.62
9995	CA-2027-119914	Keshav Sharma	243.16
9996	CA-2027-121258	Rahul Sharma	129.6
9997	CA-2027-121258	Anam Verma	258.576

Conclusion

This assignment successfully demonstrated incremental data processing using **Delta Lake** in **Apache Spark (PySpark)**. The Sample Superstore dataset was loaded, cleaned, and stored as a Delta Table. An incremental dataset containing updated and new customer records was created, and the **MERGE** operation was used to update existing records while inserting new ones. Finally, the output was validated to ensure that all changes were applied correctly without creating duplicate **Row_ID** values. This assignment provided practical experience with Delta Lake's powerful data management capabilities and real-world incremental data processing workflows.

Key Learning Outcomes

- Learned the fundamentals of **Delta Lake** and its advantages over traditional data storage.
- Loaded and processed datasets efficiently using **PySpark**.
- Performed data cleaning by removing duplicate records and validating the dataset.
- Converted data types to ensure compatibility with Delta Lake operations.
- Created and managed **Delta Tables** in Databricks.
- Built an incremental dataset containing both updated and newly inserted records.
- Applied the **Delta Lake MERGE** operation for efficient upsert functionality.
- Validated the final dataset to ensure data consistency and eliminate duplicate records.
- Gained hands-on experience with **incremental data processing** and **Delta Lake workflows** in Databricks.